

E-Commerce Platform Documentation

Project Information

Project Name: E-Commerce Platform

Author: Mohamed Makram

Repository: github.com/mohammed-makram/ecommerce-platform

Contact: mohamed@example.com

Version: 1.0.0

Table of Contents

1. [Introduction](#)
 2. [Architecture Overview](#)
 3. [Project Structure](#)
 4. [Microservices](#)
 5. [Technology Stack](#)
 6. [Prerequisites](#)
 7. [Local Development Setup](#)
 8. [Kubernetes Deployment](#)
 9. [Argo CD Configuration](#)
 10. [CI/CD Pipeline](#)
 11. [Accessing the Application](#)
 12. [Troubleshooting](#)
 13. [Production Considerations](#)
-

Introduction

This project is a comprehensive e-commerce platform developed as a DevOps professional project. It demonstrates modern cloud-native practices including microservices architecture, containerization, orchestration with Kubernetes, and GitOps-based continuous deployment

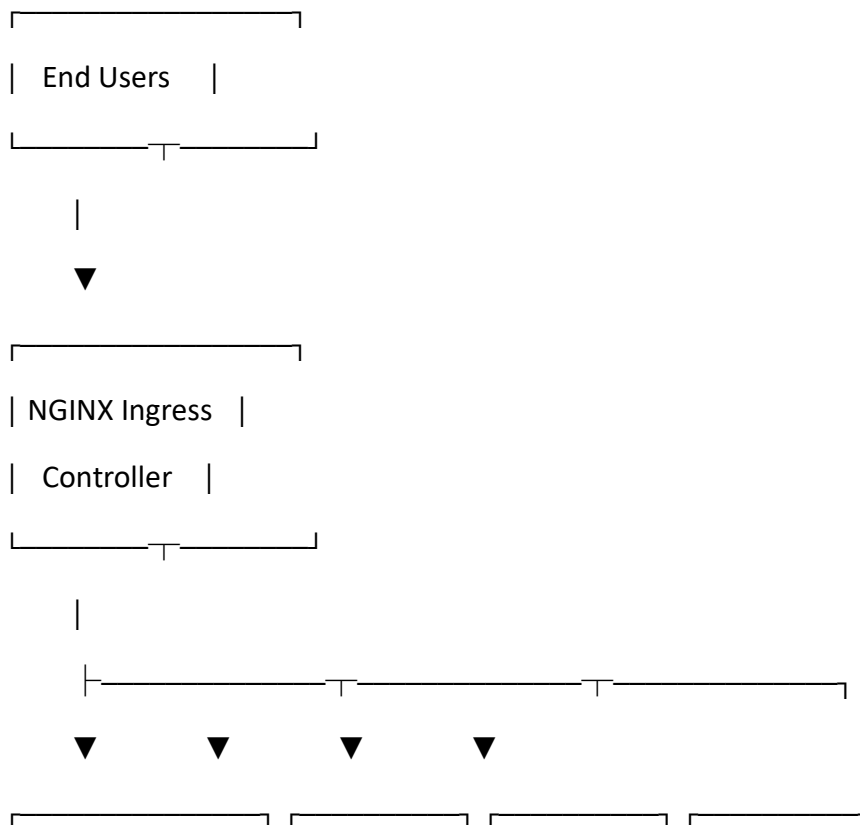
using Argo CD. The platform showcases industry best practices for building, deploying, and managing scalable applications in a production-like environment.

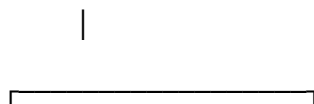
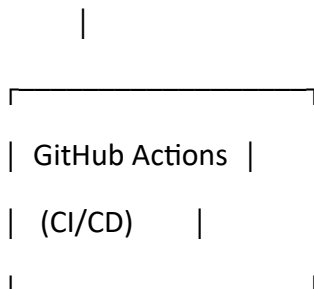
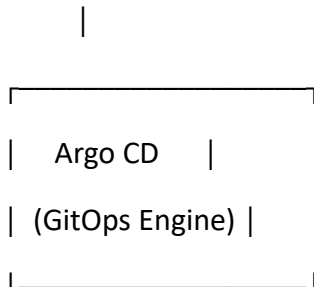
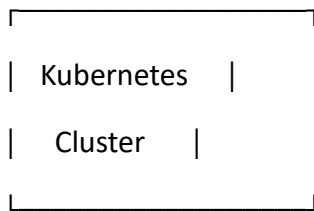
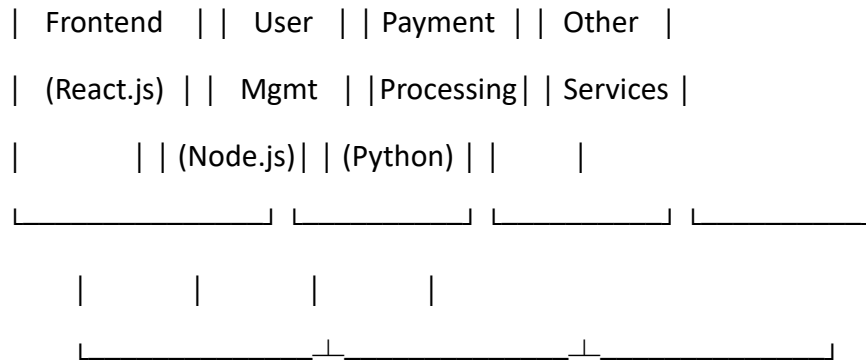
Key Features

- Microservices-based architecture for scalability and maintainability
 - Containerized services using Docker
 - Kubernetes orchestration for high availability
 - GitOps workflow with Argo CD for automated deployments
 - CI/CD pipeline using GitHub Actions
 - Infrastructure as Code with Helm charts
 - API Gateway pattern with NGINX Ingress Controller
-

Architecture Overview

The platform follows a microservices architecture pattern with the following components:





Project Structure

ecommerce-platform/

- └─ frontend/ # React.js frontend application
- | └─ src/ # Source code
- | └─ public/ # Static assets
- | └─ package.json # Node dependencies
- | └─ Dockerfile # Frontend container image
- |
- └─ backend-py/ # Python backend for payment processing
- | └─ app.py # Flask application
- | └─ requirements.txt # Python dependencies
- | └─ Dockerfile # Backend container image
- |
- └─ backend-json/ # JSON-based backend service
- | └─ [service files]
- |
- └─ .github/
- | └─ workflows/
- | └─ deploy-argo.yml # GitHub Actions CI/CD workflow
- |
- └─ Chart.yaml # Helm chart metadata
- └─ values.yaml # Helm chart configuration values

```
├── templates/           # Kubernetes resource templates
|   ├── deployment.yaml  # Deployment manifests
|   ├── service.yaml     # Service manifests
|   ├── ingress.yaml     # Ingress configuration
|   └── configmap.yaml   # Configuration maps
|
├── output.yaml          # Rendered Helm manifest
├── argocd-repo-server-backup.yaml # Argo CD repository server backup
└── README.md            # Project overview
```

Microservices

1. User Management API

Purpose: Handles user authentication, registration, and profile management.

- **Technology:** Node.js with Express
- **API Endpoint:** /api/users
- **Health Check:** /health
- **Port:** 3000
- **Features:**
 - User registration and login
 - JWT-based authentication
 - Profile management
 - Session handling

2. Payment Processing API

Purpose: Manages payment transactions and order processing.

- **Technology:** Python with Flask
- **API Endpoint:** /payment

- **Health Check:** /health
- **Port:** 5000
- **Features:**
 - Payment gateway integration
 - Transaction processing
 - Order validation
 - Payment status tracking

3. Shopping Cart Frontend

Purpose: User interface for the e-commerce platform.

- **Technology:** React.js
- **Health Check:** /
- **Port:** 80
- **Features:**
 - Product browsing
 - Shopping cart management
 - User authentication UI
 - Checkout process
 - Responsive design

Technology Stack

Frontend

- **React.js** - UI framework
- **Node.js** - Runtime environment
- **Nginx** - Web server

Backend

- **Node.js** - User management service

- **Python Flask** - Payment processing service
- **Express.js** - API framework

Infrastructure

- **Docker** - Containerization
- **Kubernetes** - Container orchestration
- **Minikube** - Local Kubernetes cluster
- **Helm** - Kubernetes package manager
- **Argo CD** - GitOps continuous delivery

CI/CD

- **GitHub Actions** - Automation pipeline
- **GitHub Container Registry** - Container image storage

Networking

- **NGINX Ingress Controller** - API Gateway and routing

Prerequisites

Before setting up the project, ensure you have the following installed:

Required Software

1. **Docker** (version 20.10 or higher)
2. `docker --version`
3. **Minikube** (version 1.25 or higher)
4. `minikube version`
5. **kubectl** (version 1.23 or higher)
6. `kubectl version --client`
7. **Helm** (version 3.8 or higher)
8. `helm version`
9. **Argo CD CLI** (version 2.5 or higher)

10. `argocd version --client`

System Requirements

- **CPU:** Minimum 2 cores (4 cores recommended)
 - **Memory:** Minimum 4GB RAM (8GB recommended)
 - **Disk Space:** 20GB free space
 - **Operating System:** Linux, macOS, or Windows with WSL2
-

Local Development Setup

Step 1: Clone the Repository

```
git clone https://github.com/mohammed-makram/ecommerce-platform.git
cd ecommerce-platform
```

Step 2: Start Minikube

Start Minikube with sufficient resources:

```
minikube start --cpus=2 --memory=3072 --driver=docker
```

Verify the cluster is running:

```
kubectl cluster-info
```

```
kubectl get nodes
```

Step 3: Enable Required Addons

Enable the NGINX Ingress Controller:

```
minikube addons enable ingress
```

Verify ingress is enabled:

```
kubectl get pods -n ingress-nginx
```

Step 4: Configure Local DNS

Get your Minikube IP address:

```
minikube ip
```


Add the following entry to your `/etc/hosts` file (Linux/Mac) or `C:\Windows\System32\drivers\etc\hosts` (Windows):

```
<MINIKUBE_IP> ecommerce.example.com
```

Example:

```
192.168.49.2 ecommerce.example.com
```

Step 5: Build Docker Images

If building locally, set Docker to use Minikube's Docker daemon:

```
eval $(minikube docker-env)
```

Build the images:

```
# Frontend
```

```
cd frontend
```

```
docker build -t ecommerce-frontend:latest .
```

```
# Python Backend
```

```
cd ../backend-py
```

```
docker build -t ecommerce-payment:latest .
```

```
# Return to root
```

```
cd ..
```

Kubernetes Deployment

Deployment Architecture

Each service is deployed with:

- **Deployment:** Manages pod replicas and rolling updates
- **Service:** Provides stable network endpoint
- **ConfigMap:** Stores configuration data

- **Ingress:** Routes external traffic

Manual Deployment Using Helm

Install the application using Helm:

```
helm install ecommerce-platform . -f values.yaml
```

Verify the deployment:

```
kubectl get pods
```

```
kubectl get services
```

```
kubectl get ingress
```

Check pod logs:

```
kubectl logs -l app=frontend
```

```
kubectl logs -l app=user-management
```

```
kubectl logs -l app=payment-processing
```

Updating the Deployment

To update the deployment with new values:

```
helm upgrade ecommerce-platform . -f values.yaml
```

Uninstalling

To remove the deployment:

```
helm uninstall ecommerce-platform
```

Argo CD Configuration

Installing Argo CD

Install Argo CD in your cluster:

```
kubectl create namespace argocd
```

```
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Accessing Argo CD UI

Port forward the Argo CD server:

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

Access the UI at: <https://localhost:8080>

Getting Initial Admin Password

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" |  
base64 -d
```

Logging in with CLI

```
argocd login localhost:8080 --username admin --password <password>
```

Creating the Application

Create the Argo CD application:

```
argocd app create ecommerce-platform \  
--repo https://github.com/mohammed-makram/ecommerce-platform.git \  
--path . \  
--dest-server https://kubernetes.default.svc \  
--dest-namespace default
```

Syncing the Application

Manually sync the application:

```
argocd app sync ecommerce-platform
```

Check application status:

```
argocd app get ecommerce-platform
```

CI/CD Pipeline

GitHub Actions Workflow

The CI/CD pipeline is defined in `.github/workflows/deploy-argo.yml` and performs the following:

1. **Trigger:** Activates on push to master branch
2. **Authentication:** Logs into Argo CD using GitHub secrets

3. **Sync:** Triggers application sync in Argo CD
4. **Verification:** Checks deployment status

Required GitHub Secrets

Configure the following secrets in your GitHub repository:

- ARGOCD_SERVER: Argo CD server URL
- ARGOCD_USERNAME: Argo CD admin username
- ARGOCD_PASSWORD: Argo CD admin password

Setting Up Secrets

1. Navigate to your GitHub repository
2. Go to Settings → Secrets and variables → Actions
3. Click "New repository secret"
4. Add each secret with its corresponding value

Workflow Configuration

name: Deploy to Argo CD

on:

push:

branches:

- master

jobs:

deploy:

runs-on: ubuntu-latest

steps:

- name: Checkout code

uses: actions/checkout@v2

- name: Login to Argo CD

run: |

```
argocd login ${{ secrets.ARG OCD_SERVER }} \  
--username ${{ secrets.ARG OCD_USERNAME }} \  
--password ${{ secrets.ARG OCD_PASSWORD }} \  
--insecure
```

- name: Sync application

run: argocd app sync ecommerce-platform

Accessing the Application

Local Access

Once deployed, access the application at:

<http://ecommerce.example.com>

Service Endpoints

- **Frontend:** <http://ecommerce.example.com/>
- **User API:** <http://ecommerce.example.com/api/users>
- **Payment API:** <http://ecommerce.example.com/payment>

Health Checks

Verify services are healthy:

```
curl http://ecommerce.example.com/health
```

```
curl http://ecommerce.example.com/api/users/health
```

```
curl http://ecommerce.example.com/payment/health
```

Monitoring

View real-time logs:

```
# All pods
```

```
kubectl logs -f -l app=ecommerce
```

```
# Specific service
```

```
kubectl logs -f -l app=frontend
```

Troubleshooting

Common Issues and Solutions

Pods Not Starting

Problem: Pods stuck in Pending or CrashLoopBackOff state

Solution:

```
# Check pod status
```

```
kubectl describe pod <pod-name>
```

```
# Check logs
```

```
kubectl logs <pod-name>
```

```
# Check resource availability
```

```
kubectl top nodes
```

Ingress Not Working

Problem: Cannot access application via domain

Solution:

```
# Verify ingress is running
```

```
kubectl get ingress
```

```
kubectl describe ingress ecommerce-ingress
```

```
# Check ingress controller
```

```
kubectl get pods -n ingress-nginx
```

```
# Verify /etc/hosts entry
```

```
cat /etc/hosts | grep ecommerce
```

Image Pull Errors

Problem: ImagePullBackOff or ErrImagePull

Solution:

```
# Use Minikube's Docker daemon
```

```
eval $(minikube docker-env)
```

```
# Rebuild images
```

```
docker build -t <image-name>:latest .
```

```
# Verify images exist
```

```
docker images | grep ecommerce
```

Argo CD Sync Fails

Problem: Application out of sync or sync fails

Solution:

```
# Check application status
```

```
argocd app get ecommerce-platform
```

```
# View sync history
```

```
argocd app history ecommerce-platform
```

```
# Force refresh
```

```
argocd app sync ecommerce-platform --force
```

Production Considerations

Security

1. **Enable TLS/SSL:** Use cert-manager with Let's Encrypt
2. **Secrets Management:** Use Sealed Secrets or external secret managers
3. **Network Policies:** Implement pod-to-pod communication restrictions
4. **RBAC:** Configure proper role-based access control
5. **Image Scanning:** Integrate vulnerability scanning in CI/CD

Scalability

1. **Horizontal Pod Autoscaler:** Auto-scale based on CPU/memory
2. **Persistent Storage:** Replace in-memory storage with databases
3. **Caching Layer:** Implement Redis for session and data caching
4. **Load Balancing:** Use external load balancer for production traffic
5. **CDN Integration:** Serve static assets via CDN

Monitoring and Observability

1. **Prometheus:** Metrics collection
2. **Grafana:** Visualization dashboards
3. **ELK Stack:** Centralized logging
4. **Jaeger:** Distributed tracing
5. **AlertManager:** Alert notification system

High Availability

1. **Multi-replica Deployments:** Run multiple pod replicas
2. **Pod Disruption Budgets:** Ensure minimum availability during updates
3. **Health Checks:** Configure liveness and readiness probes
4. **Backup Strategy:** Regular backup of configurations and data
5. **Disaster Recovery:** Document and test recovery procedures

Performance Optimization

1. **Resource Limits:** Set appropriate CPU and memory limits
 2. **Database Optimization:** Index frequently queried fields
 3. **API Rate Limiting:** Use persistent backend (Redis) for rate limiting
 4. **Caching Strategy:** Implement multi-level caching
 5. **Connection Pooling:** Optimize database connections
-

License

This project is licensed under the MIT License - see the LICENSE file for details.

Acknowledgments

- Kubernetes community for excellent documentation
 - Argo CD team for GitOps best practices
 - DevOps community for inspiration and support
-

Changelog

Version 1.0.0 (Initial Release)

- Microservices architecture implementation
 - Kubernetes deployment configuration
 - Argo CD GitOps integration
 - GitHub Actions CI/CD pipeline
 - Local development with Minikube
-

Last Updated: October 2025

Maintained By: Mohamed Makram

Status: Active Development